

# Week 6 - The transform system

ROBOTIC EMBEDDED PROGRAMMING  
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.



**WE ARE  
HUMBER**

# Review from Last Week



- **Robots need math:** coordinate transforms = foundation for control, planning, and simulation
- **Points in space:** use vectors (2D/3D) to represent robot geometry
- **Transformations:** functions map points to new positions/orientations
- **Linear framework:** matrices capture rotation, scaling, shear; properties simplify chaining
- **2D & 3D rotations:** derived from trig, preserve distance & orientation ( $SO(2)$ ,  $SO(3)$ )
- **Translations:** non-linear in raw form, solved via **homogeneous coordinates**
- **Unified representation:** rotation + translation together → single matrix multiply ( $SE(2)/SE(3)$ )
- Sets the stage for: **frames, kinematics, and full robot motion analysis**

# Agenda for today's training session



## Performance Objective:

- Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
- Configure ROS2 on a Linux-based system
- Develop ROS2 nodes in Python or C++ using rclpy or rclcpp
- Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices
- Utilize ROS2 topics, services and actions for inter-node communication
- Implement coordinate frame management using the tf2 library to track and transform between reference frames in robotic applications

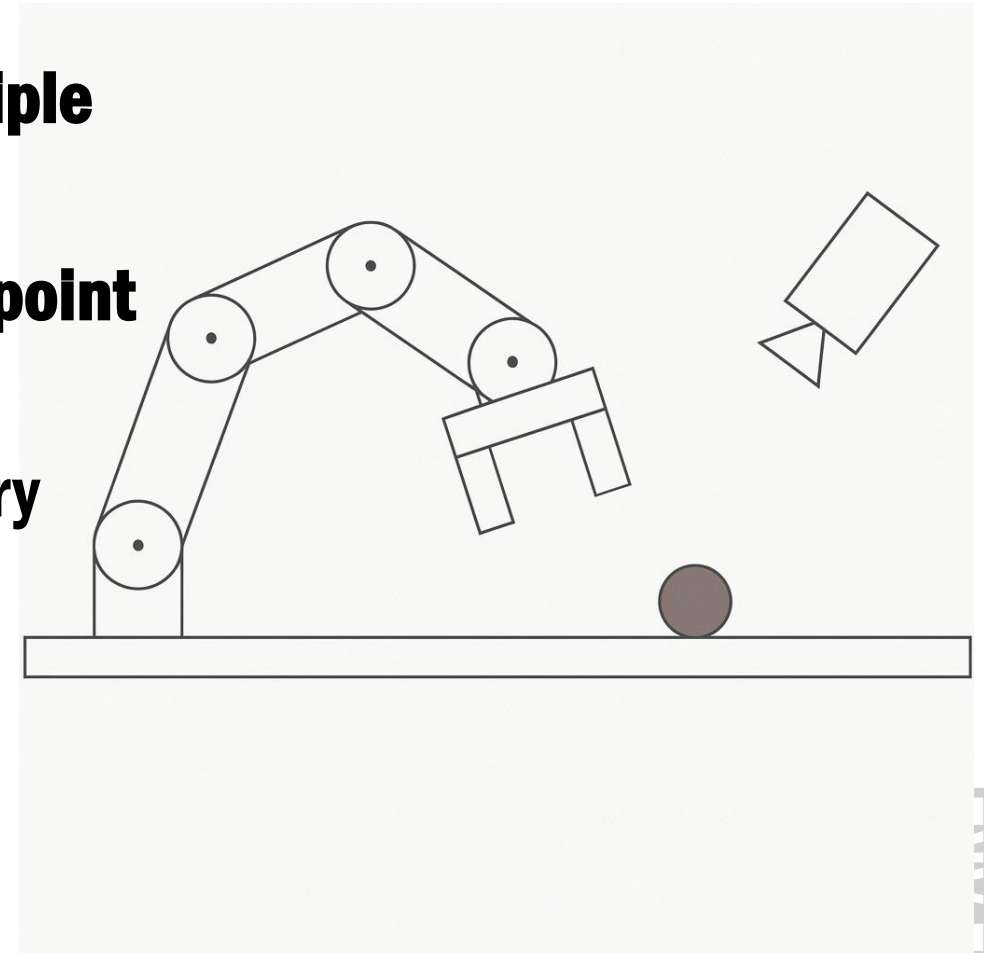
## Problem we will be solving today:

- Transforms in ROS
- Broadcasting single frames
- Viewing frames and transforms in RVIZ
- 3 – Broadcasting a moving robot

# Why Are Transforms Important?

## The Transform System (tf2): Why Transforms Matter

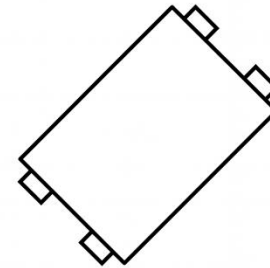
- **Robots interact with the world through multiple perspectives**
- **Needed to convert measurements from one point of view to another**
- **Prevents repetitive and complex trigonometry (esp. in 3D)**
- **Foundation for localization, mapping, perception, and control**



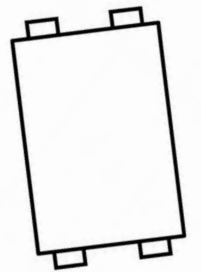
# Example 1: Multi-Robot Exploration

## The Transform System (tf2): Multi-Robot Exploration

- Robot A finds an object
- Robot B needs to navigate to it
- Requires a **shared frame of reference**
- Transforms allow Robot B to understand the object's position



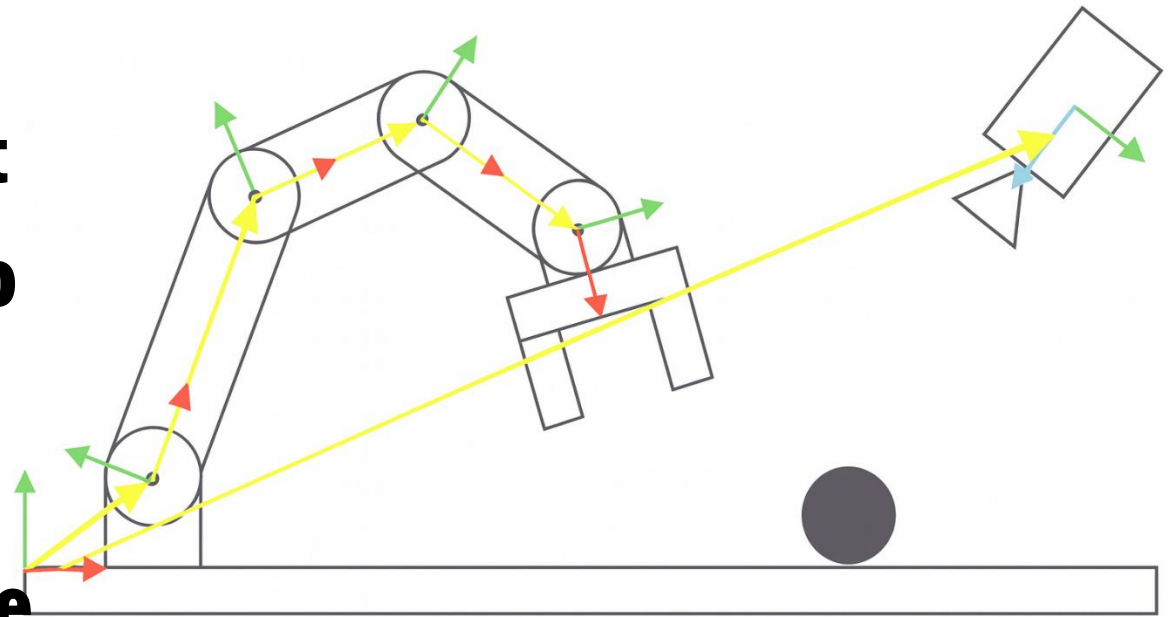
[source](#)



## Example 2: Camera & Manipulator

### The Transform System (tf2): Camera & Manipulator

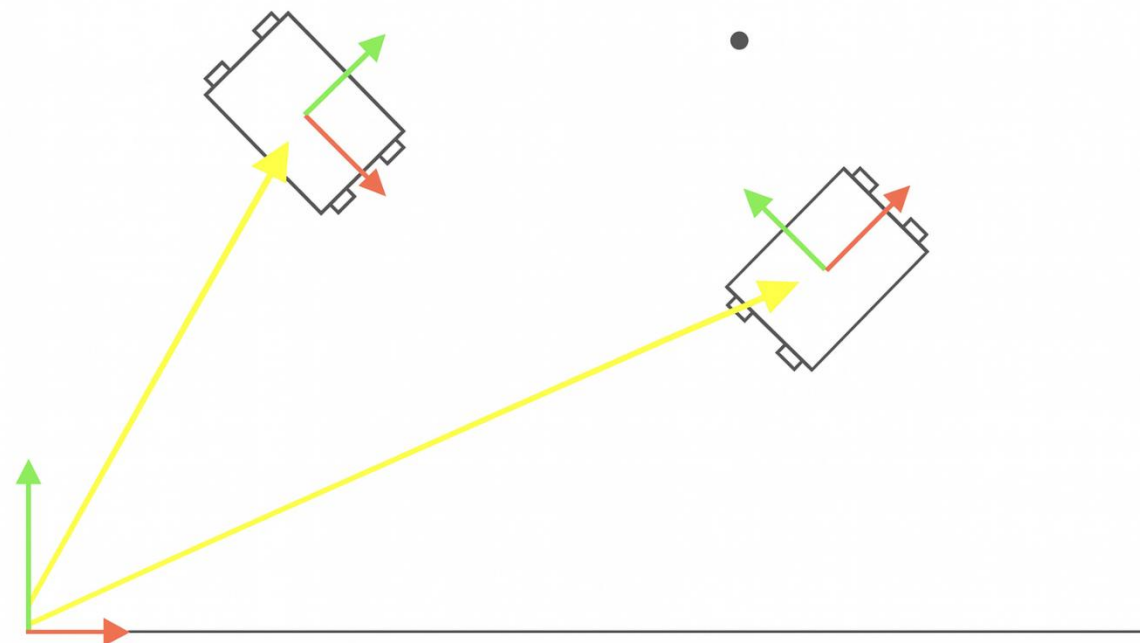
- Mounted camera detects a target
- Manipulator must move **gripper to target**
- Transforms link **camera frame** → **robot base frame** → **gripper frame**



# Frames & Transforms

## The Transform System (tf2): Frames and Relationships

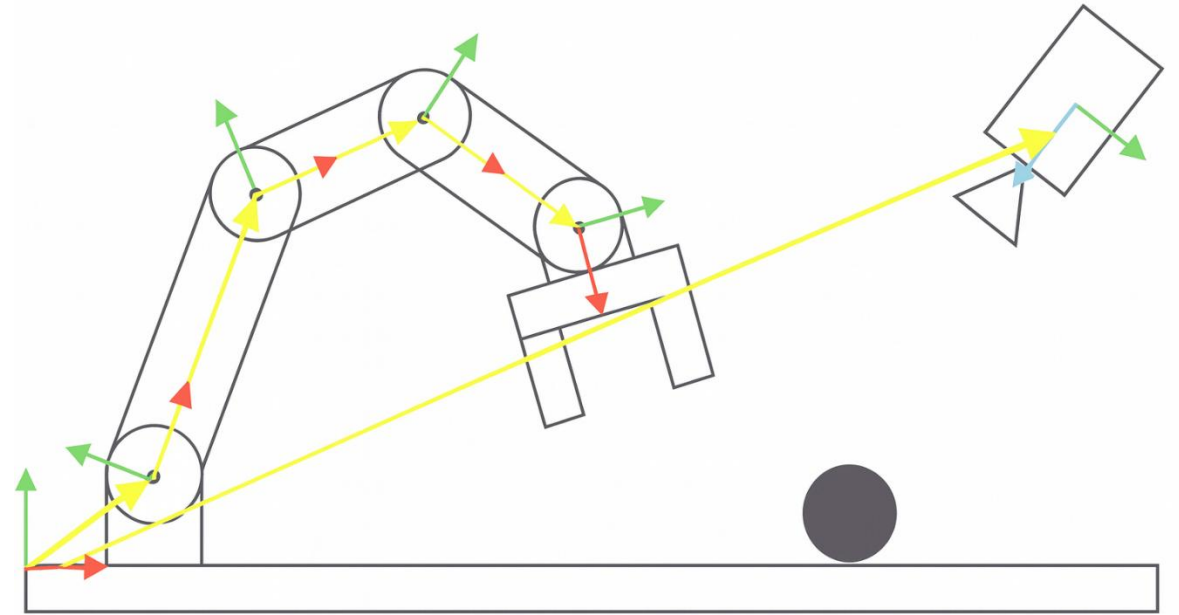
- **Frame:** Coordinate system tied to a robot part or sensor
- **Transform:** Translation + Rotation between frames
- **Bidirectional:** can convert both ways
- **Example:** From camera frame to world frame



# Transform Tree

## The Transform System (tf2): Transform Tree

- Each frame linked to **only one parent frame**
- Creates a **tree structure**
- Any point in one frame can be converted to another
- Enables modular and scalable robot systems



# ROS 2 tf2 Subsystem

## The Transform System (tf2): ROS 2 Tools

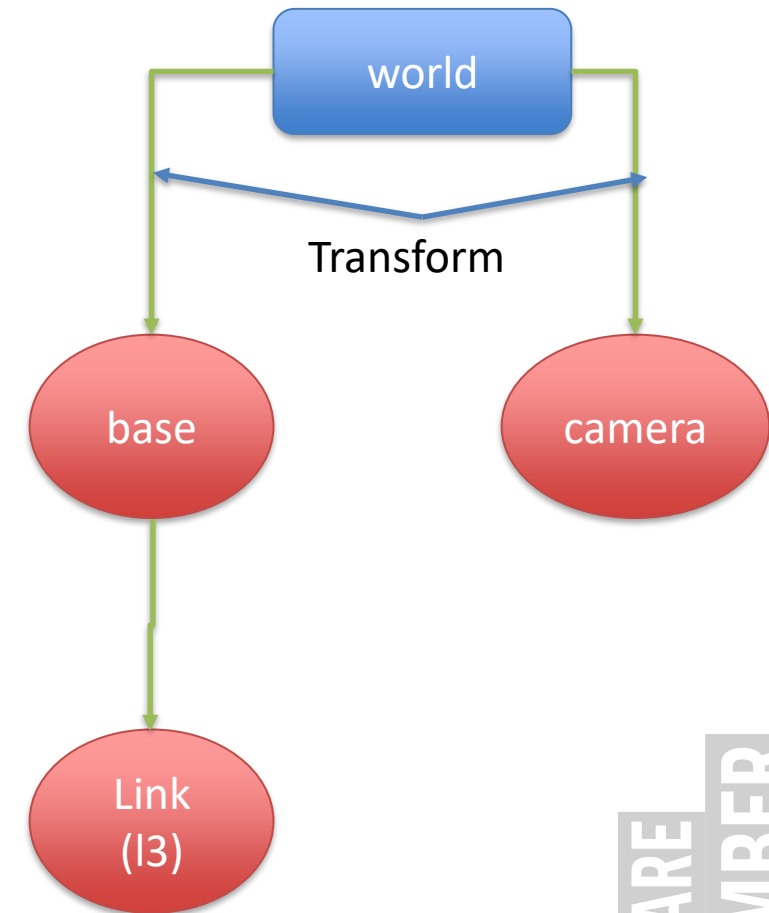
- **Handles all the math + bookkeeping for you**
- **Maintains transform tree automatically**
- **Provides tools:**
  - `tf2` **library (C++/Python APIs)**
  - `tf2_ros` **(ROS interfaces)**
  - **Visualization in RViz**

# Transform System (tf2): Transforms in ROS

- ROS uses **tf2** to manage coordinate transformations
- Any node can:
  - **Broadcast** a transform (frame → frame)
  - **Listen** for transforms and convert points between frames

# The Transform System (tf2): Tree Structure

- Transforms must form a **tree**
- Each frame has **one parent** but can have many children
- Example tree:
  - `base` & `camera` relative to `world`
  - `l3` relative to `base`
- Allows conversion between any two connected frames

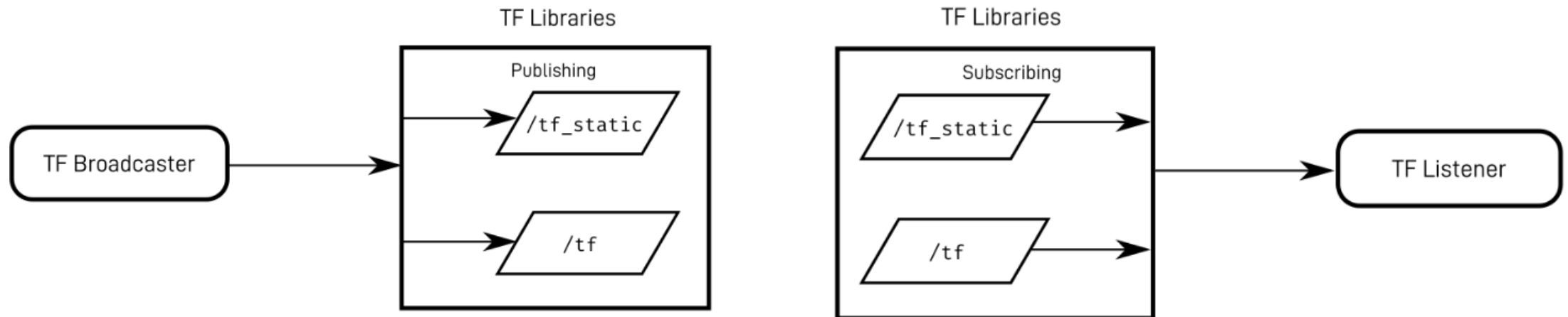


# The Transform System (tf2): Static vs Dynamic Transforms

- **Static transforms**
  - Don't change over time
  - Broadcast once, remain valid
- **Dynamic transforms**
  - Can change over time
  - Must be updated regularly
  - System flags error if updates stop

# The Transform System (tf2): Communication

- Uses ROS topics internally:
  - `/tf` → for dynamic transforms
  - `/tf_static` → for static transforms
- Called **broadcasting & listening** (not publishing/subscribing)



# The Transform System (tf2): Built-in Support

- We can write nodes to broadcast transforms
- But ROS provides **built-in nodes** for common tasks
- Helpful for beginners → no need to calculate transforms manually

## tf2: Broadcasting Single Frames

- Use `static_transform_publisher` to broadcast static transforms
- Limitations:
  - Cannot broadcast dynamic transforms
  - Still very useful for:
    - Learning & testing transforms
    - Acting as “glue” between nodes expecting different frames

## tf2: Static Transform Command

```
ros2 run tf2_ros static_transform_publisher \ x y z yaw pitch roll parent_frame child_frame
```

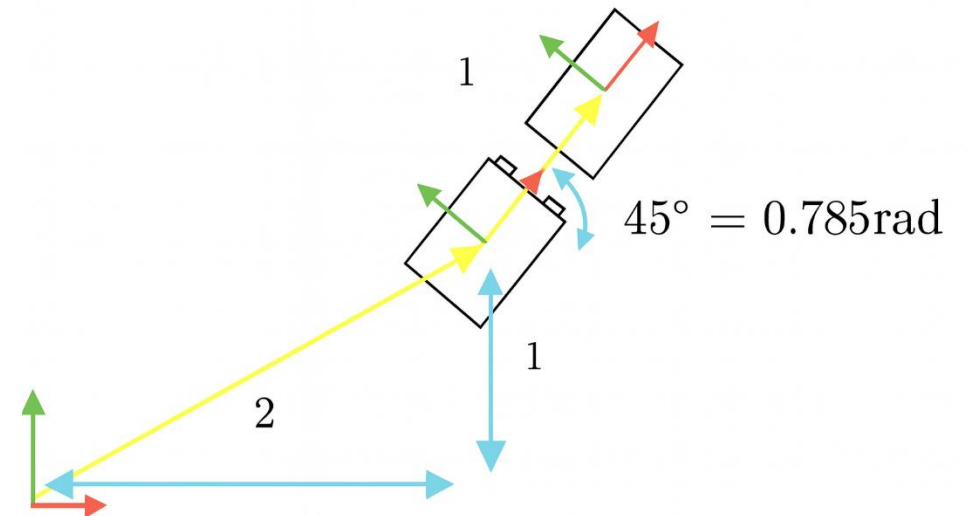
- Translation: `x y z`
- Rotation: `yaw pitch roll` (radians)
- Order: **translations first, then rotations**
- Rotations applied with respect to **local coordinate system**

## tf2: Example – Adding robot\_1

- Example command:

```
ros2 run tf2_ros static_transform_publisher \ 2 1 0 0.785 0 0 world robot_1
```

- Frame `robot_1` placed:
  - 2m in x, 1m in y, 0m in z
  - Rotated  $45^\circ$  (0.785 rad) around z-axis
- Defined relative to `world`



## tf2: Example – Adding robot\_2 (Attached to robot 1)

- Command in second terminal:

```
ros2 run tf2_ros static_transform_publisher \ 1 0 0 0 0 0 robot_1 robot_2
```

- Frame `robot_2` sits:
  - 1m in the +x direction of `robot_1`
- Creates a sidecar setup

# tf2: Viewing Frames and Transforms in RViz

- In ROS 2, visualization tool = rviz2
- Run with:

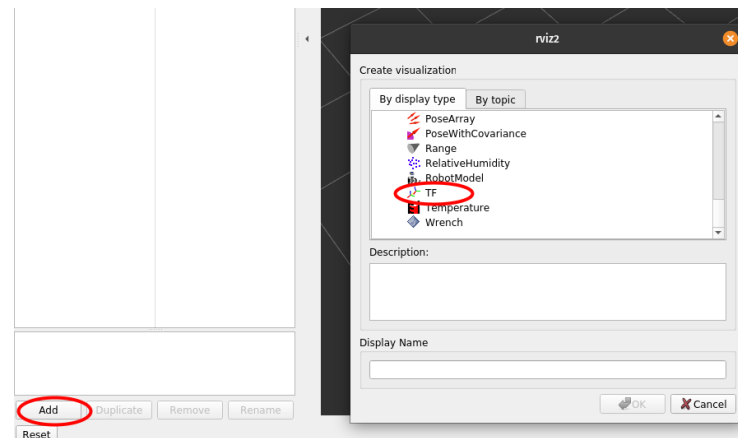
```
ros2 run rviz2 rviz2
```

- Or simply:

```
rviz2
```

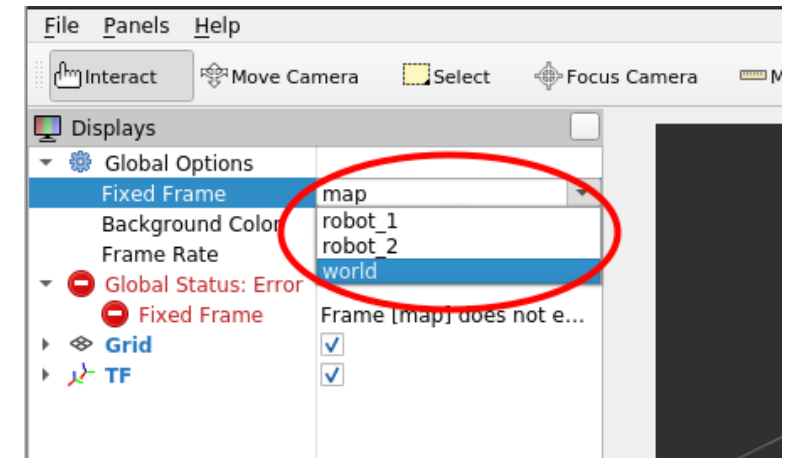
## tf2: Adding TF Data in RViz

- In RViz:
  - Click **Add** (bottom-left corner)
  - Select **TF**
- Initially, transforms won't appear because no reference frame is set



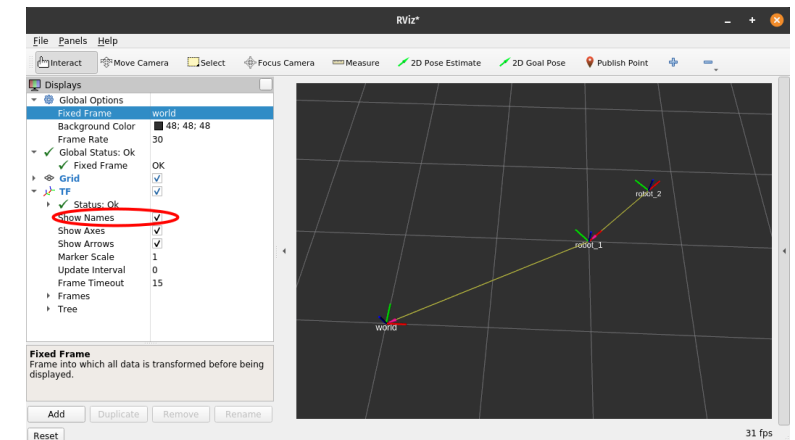
## tf2: Fixing the Reference Frame

- Default fixed frame = **map** (not available)
- Change fixed frame (top-left) → **world**
- Then:
  - **Global Status** = healthy
  - Frames appear with connecting lines



# tf2: Configuring TF Display in RViz

- Use left panel settings to customize display
- Options include:
  - Enable showing **frame names**
  - Adjust visibility of transforms



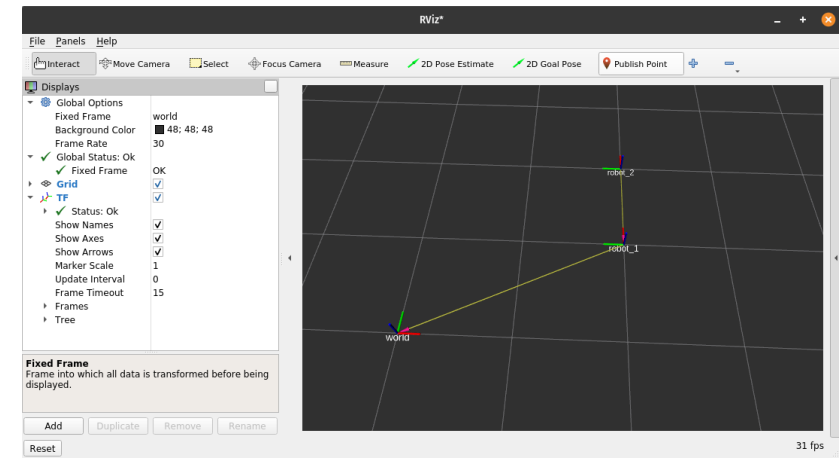
⚠ Note: RViz arrows show **child** → **parent**

## tf2: Static Transform Example (Rotation)

- Lets go back to our first terminal, and rerun the command with a larger rotation of  $90^\circ$  (1.57 rad):

```
ros2 run tf2_ros static_transform_publisher \ 2 1 0 1.57 0 0 world robot_1
```

- Rotates robot\_1 by  $90^\circ$  (1.57 rad)
- robot\_2 also moves (since it's relative to robot\_1)



## tf2: Experimenting in RViz

- Try:
  - Changing **fixed frame** to a robot frame
  - Adjusting translations & rotations
  - Adding more frames
- Observe how transforms propagate

## tf2: Cleanup Before Next Step

- Stop all `static_transform_publishers`
- Close RViz
- Prevents interference in future experiments

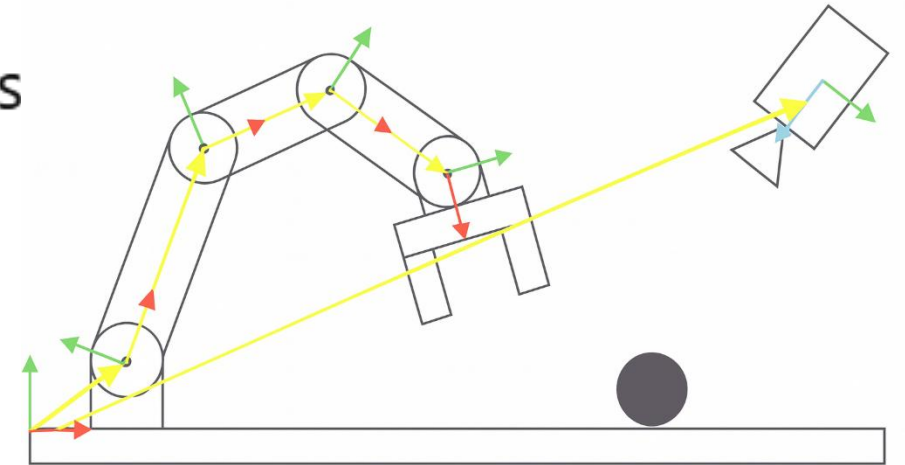
## tf2: Broadcasting a Moving Robot

- **Until now: only static transforms**
- **Next step: dynamic transforms for moving robots**
- **Install required packages:**

```
sudo apt install ros-jazzy-xacro ros-jazzy-joint-state-publisher-gui
```

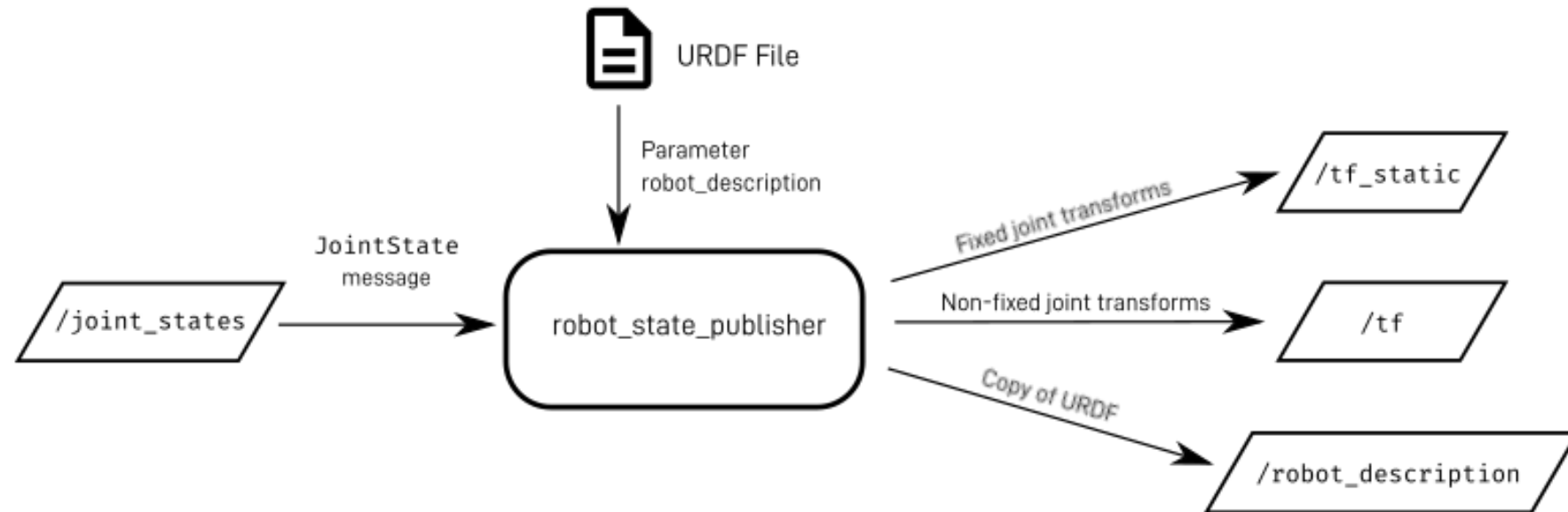
## tf2: URDF Files

- URDF = **Unified Robot Description Format**
- Defines:
  - Size, shape, and colour of robot parts
  - Links (rigid components)
  - Joints (relationships between links)
- Follows a **tree structure** (parent → child)
- Very similar to frame/transform tree



# tf2: robot\_state\_publisher

- Reads URDF and **broadcasts transforms automatically**
- Publishes URDF to `/robot_description`
- Handles:
  - Fixed joints → static transforms
  - Movable joints → dynamic transforms (needs values)

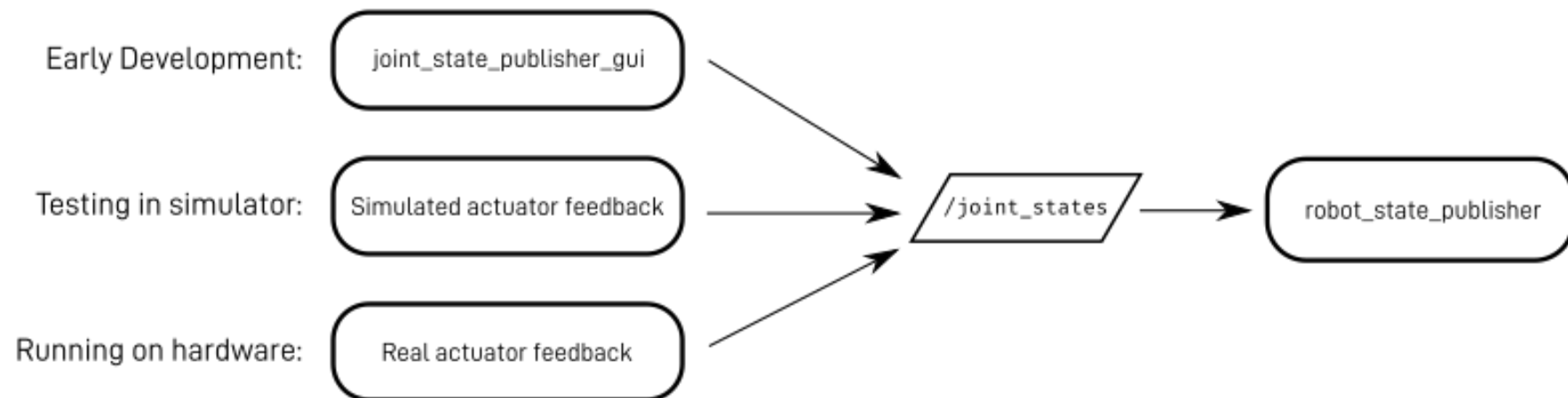


## tf2: Joint States

- Values for movable joints come from `/joint_states` topic
- Message type: `JointState`
  - Includes **position, velocity, effort**
  - We will use **position only**
- Typically sourced from:
  - **Actuator feedback sensors** (encoders, potentiometers)
  - Or simulated equivalents

## tf2: joint\_state\_publisher\_gui

- Provides sliders for each movable joint
- Reads `/robot_description`
- Publishes slider values → `/joint_states`
- Allows interactive testing without real sensors

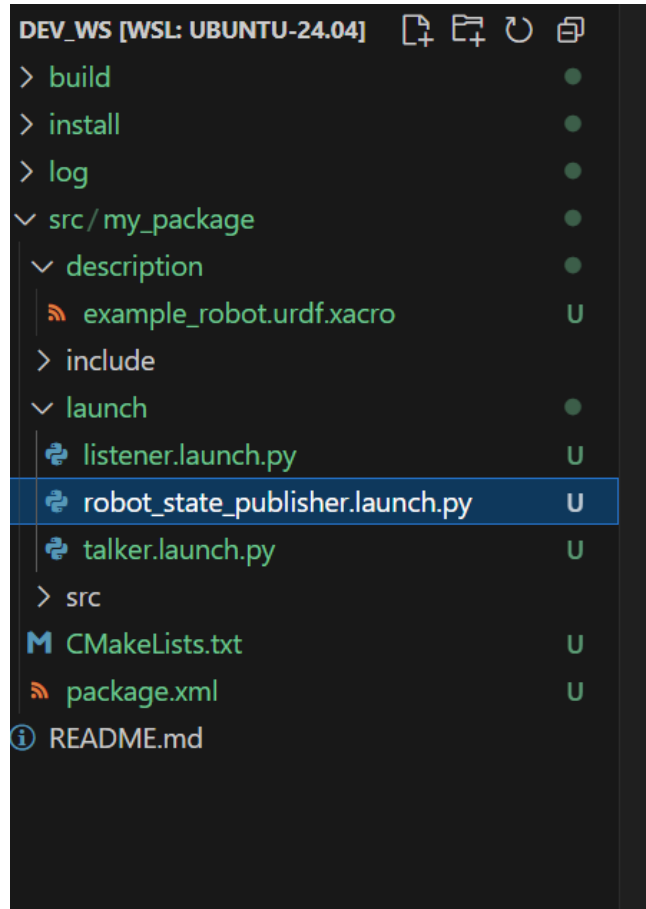


# Robot State Publisher & URDF Visualization in ROS 2

- **Objective: Connect URDF/Xacro, robot\_state\_publisher, joint\_state\_publisher\_gui, and RViz2.**
  - Write robot description in URDF/Xacro
  - Create a launch file to load URDF into ROS 2
  - Add dependencies in `package.xml`
  - Configure `CMakeLists.txt` to install launch/description files
  - Run three terminals: publisher, GUI, and RViz2
  - Visualize robot with TF + RobotModel

# Create Launch File (code file attached)

Path: `launch/robot_state_publisher.launch.py`



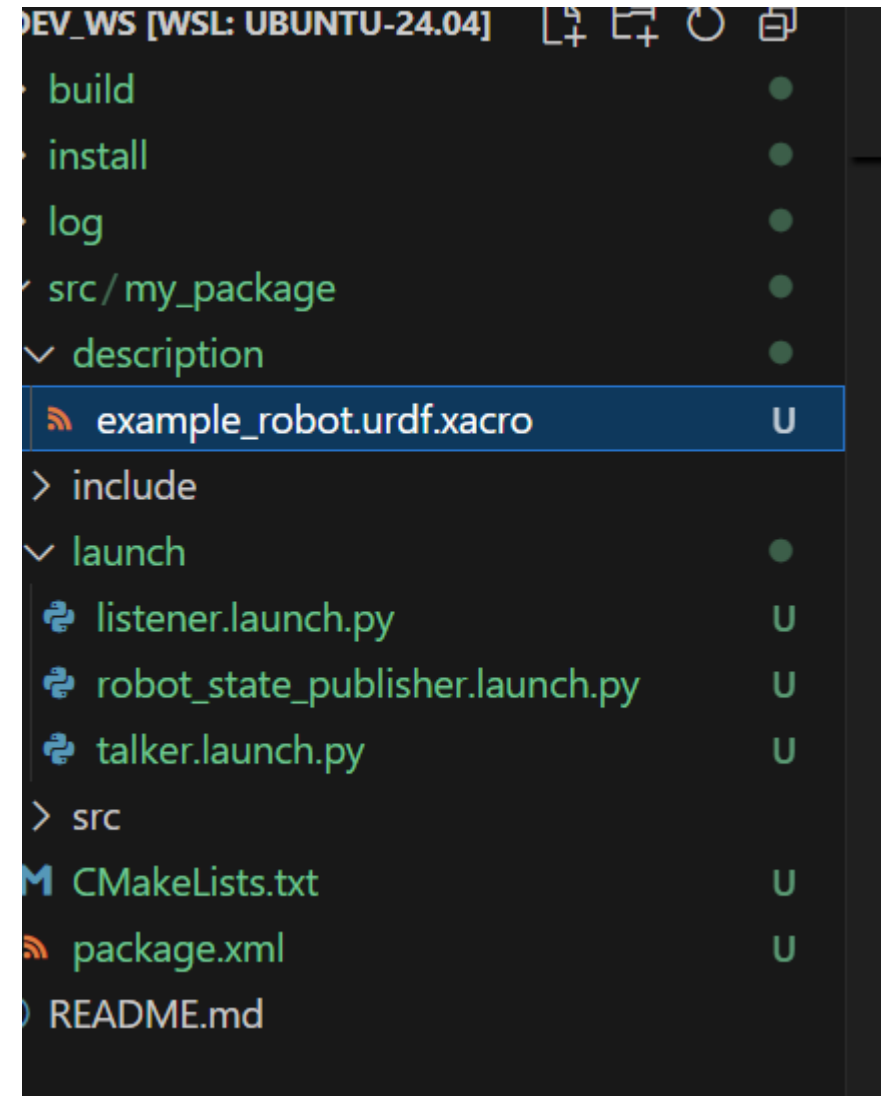
DEV\_WS [WSL: UBUNTU-24.04]

- > build
- > install
- > log
- ▼ src/my\_package
  - ▼ description
    - example\_robot.urdf.xacro
  - > include
  - ▼ launch
    - listener.launch.py
    - robot\_state\_publisher.launch.py**
    - talker.launch.py
  - > src
- CMakeLists.txt
- package.xml
- README.md

# Create URDF/Xacro File (code file attached)

Path: `description/example_robot.urdf.xacro`

Note: You need to create **description** folder first



# Update package.xml

```
<exec_depend>robot_state_publisher</exec_depend>
```

```
<exec_depend>robot_state_publisher</exec_depend>  
<exec_depend>launch</exec_depend>  
<exec_depend>launch_ros</exec_depend>  
<exec_depend>joint_state_publisher_gui</exec_depend>
```

# Update CMakeLists.txt

 Path: CMakeLists.txt

```
install(DIRECTORY launch description  
        DESTINATION share/${PROJECT_NAME}  
        )
```

# Build & Source Workspace

```
cd ~/dev_ws
```

```
colcon build --symlink-install
```

```
source install/setup.bash
```

Run Robot State Publisher :

```
ros2 launch my_package robot_state_publisher.launch.py
```

Run Joint State Publisher GUI (new terminal):

```
ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

Run RViz2 (new terminal):

```
rviz2
```

# RVIZ

- **Add TF in RViz2**
  - In RViz2 → Add → Select TF
  - Set Fixed Frame = world
- **Add Robot Model in RViz2**
  - In RViz2 → Add → Select RobotModel
  - Set Description Topic = /robot\_description

# Full Workflow Recap

- Terminal 1:

```
bash
```

```
ros2 launch my_package robot_state_publisher.launch.py
```

- Terminal 2:

```
bash
```

```
ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

# Full Workflow Recap

- Terminal 3:

```
bash  
  
rviz2
```

- In RViz2:
  - Add TF → set Fixed Frame = `world`
  - Add RobotModel → set Topic = `/robot_description`

## ⚡ Final Pipeline:

Xacro → URDF → robot\_state\_publisher → TF + `/robot_description` → RViz2 (TF + RobotModel)

# Summary



- **Transforms in ROS**
  - Handle conversions between coordinate frames
  - Organized as a **tree** (each frame has one parent)
- **Static vs Dynamic Transforms**
  - Static = fixed, broadcast once
  - Dynamic = change over time, must be updated
- **Broadcasting Frames**
  - `static_transform_publisher` → quick tests, “glue” between nodes
  - Examples: `world → robot_1 → robot_2`

# Summary



- Viewing in RViz
  - Add TF visualisation
  - Set **fixed frame** (e.g., world)
  - Arrows show **child** → **parent** relationships
  - Can configure display (frame names, robot model, etc.)
- Broadcasting a Moving Robot
  - Use **URDF** to define robot links & joints
  - `robot_state_publisher` auto-broadcasts transforms from URDF
  - `joint_state_publisher_gui` provides sliders for moving joints
  - Visualise in RViz with **TF** and **RobotModel**

THANK YOU.



**WE ARE**

**HUMBER**